Asymmetric VPP / Custom Pipeline Layout

Write down which layers each virtual stage holds

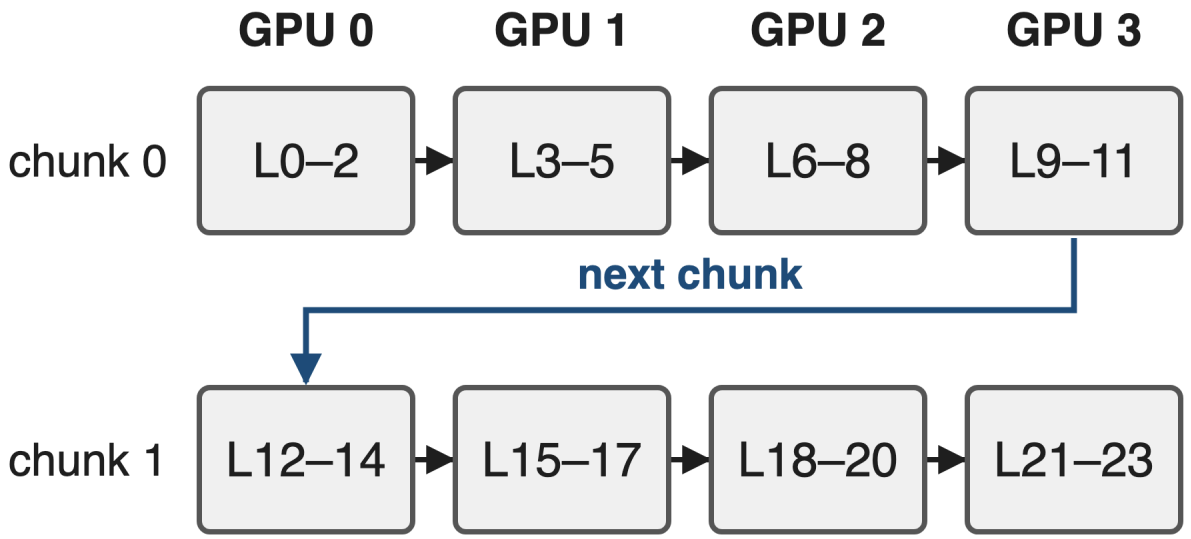
NVIDIA · Megatron-Core (v0.13+)

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

Parallelism Strategy · github.com/NVIDIA/Megatron-LM/blob/main/megatron/core/transformer/moe/README.md

Interleaving shrinks the pipeline bubble v-fold

- Each GPU holds **v** chunks of layers
- Bubble ≈ (p – 1) / (v · m)
- Worth up to **10%** throughput (PTD-P, GPT-3 175B)



24 layers · PP 4 · VPP 2 · 3 layers per chunk

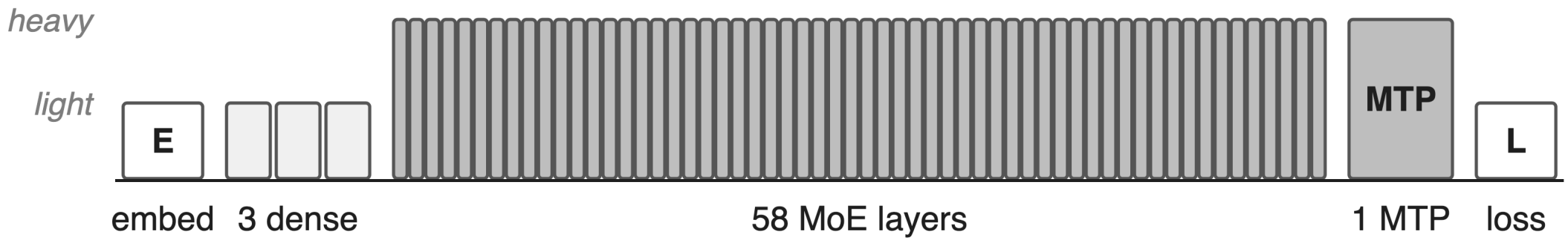
DeepSeek-V3 has 61 decoder layers: a prime

61

decoder layers, plus embedding, 1 MTP layer and loss: no uniform split over PP × VPP stages exists

Megatron-Core MoE report, arXiv 2603.07685, §7.5 and Table 10 (61 decoders + 1 MTP)

Uniform VPP splits by count, not by cost



61 decoder layers ÷ 32 virtual stages = 1.9 per stage

61 is prime: no uniform split exists

Light layers sit at both ends, heavy MoE in the middle, the heavy MTP at the tail

Old knobs only nudge the edge stages

- **Uniform VPP**
- `--num-layers-per-virtual-pipeline-stage`
- **Needs layers divisible by PP · VPP**

- **Uneven-edge flags**
- First/last stage layer counts only
- Embedding or loss = **one whole layer**

Needed: control over every stage's contents, not just its size

Insight: a pipeline is just a list of stages

- Each virtual stage holds some layers, in order
- Nothing requires equal counts

Insight: a pipeline is just a list of stages

- Each virtual stage holds some layers, in order
- Nothing requires equal counts

Let the user write the list: group light layers, isolate heavy ones, any total works

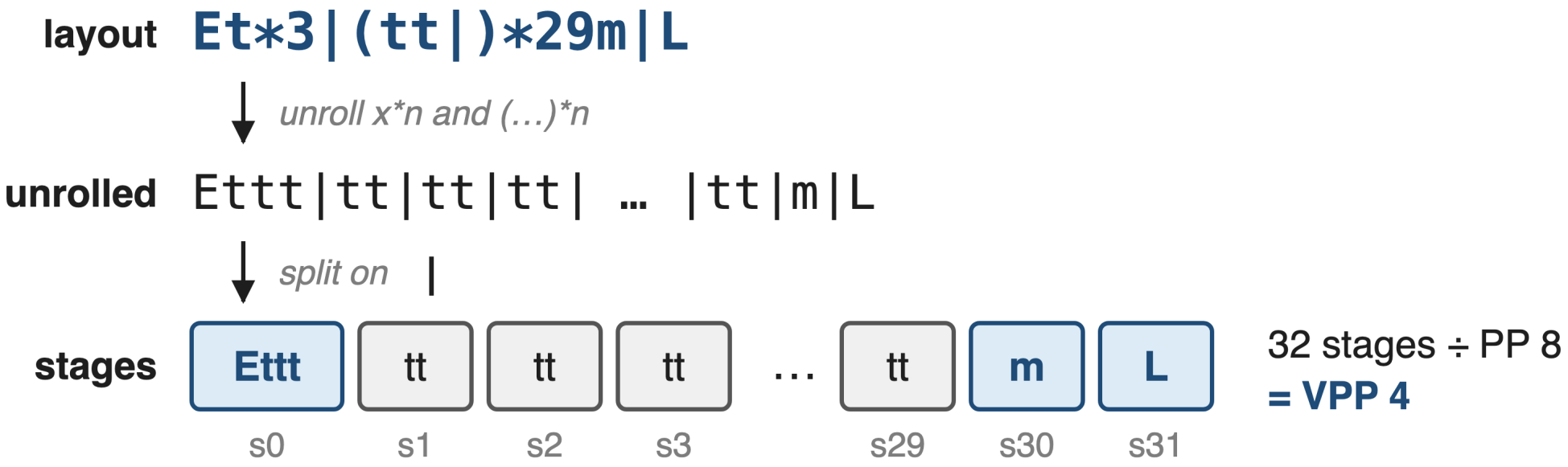
One string places every layer on every chunk

```
--pipeline-model-parallel-layout Et*3|(tt|)*29m|L
```

	PP 0	PP 1	PP 2	PP 3	PP 4	PP 5	PP 6	PP 7
chunk 0	**E+3 dense**	2 MoE	2 MoE	2 MoE	2 MoE	2 MoE	2 MoE	2 MoE
chunk 1	2 MoE	2 MoE	2 MoE	2 MoE	2 MoE	2 MoE	2 MoE	2 MoE
chunk 2	2 MoE	2 MoE	2 MoE	2 MoE	2 MoE	2 MoE	2 MoE	2 MoE
chunk 3	2 MoE	2 MoE	2 MoE	2 MoE	2 MoE	2 MoE	**MTP**	**loss**

DeepSeek-V3 on 1,024 H100 (PP 8): embedding with 3 dense layers, MTP standalone, loss separated

❶ The string unrolls into one entry per stage



E embedding · t transformer layer · m MTP layer · L loss

Commas are cosmetic; empty stages are allowed; the stage count must be a multiple of PP

❷ Stage s lands on rank s mod p

```
--pipeline-model-parallel-layout Et*3|(tt|)*29m|L
```

	PP 0	PP 1	PP 2	PP 3	PP 4	PP 5	PP 6	PP 7
chunk 0	s0 **E+3 dense**	s1 2 MoE	s2 2 MoE	s3 2 MoE	s4 2 MoE	s5 2 MoE	s6 2 MoE	s7 2 MoE
chunk 1	s8 2 MoE	s9 2 MoE	s10 2 MoE	s11 2 MoE	s12 2 MoE	s13 2 MoE	s14 2 MoE	s15 2 MoE
chunk 2	s16 2 MoE	s17 2 MoE	s18 2 MoE	s19 2 MoE	s20 2 MoE	s21 2 MoE	s22 2 MoE	s23 2 MoE
chunk 3	s24 2 MoE	s25 2 MoE	s26 2 MoE	s27 2 MoE	s28 2 MoE	s29 2 MoE	s30 **MTP**	s31 **loss**
	E+3d+6M	8 MoE	8 MoE	8 MoE	8 MoE	8 MoE	**6M+MTP**	**6M+loss**

Per-rank totals: d = dense layer, M = MoE layer (derived from the layout string)

❸ Group light layers, isolate heavy ones

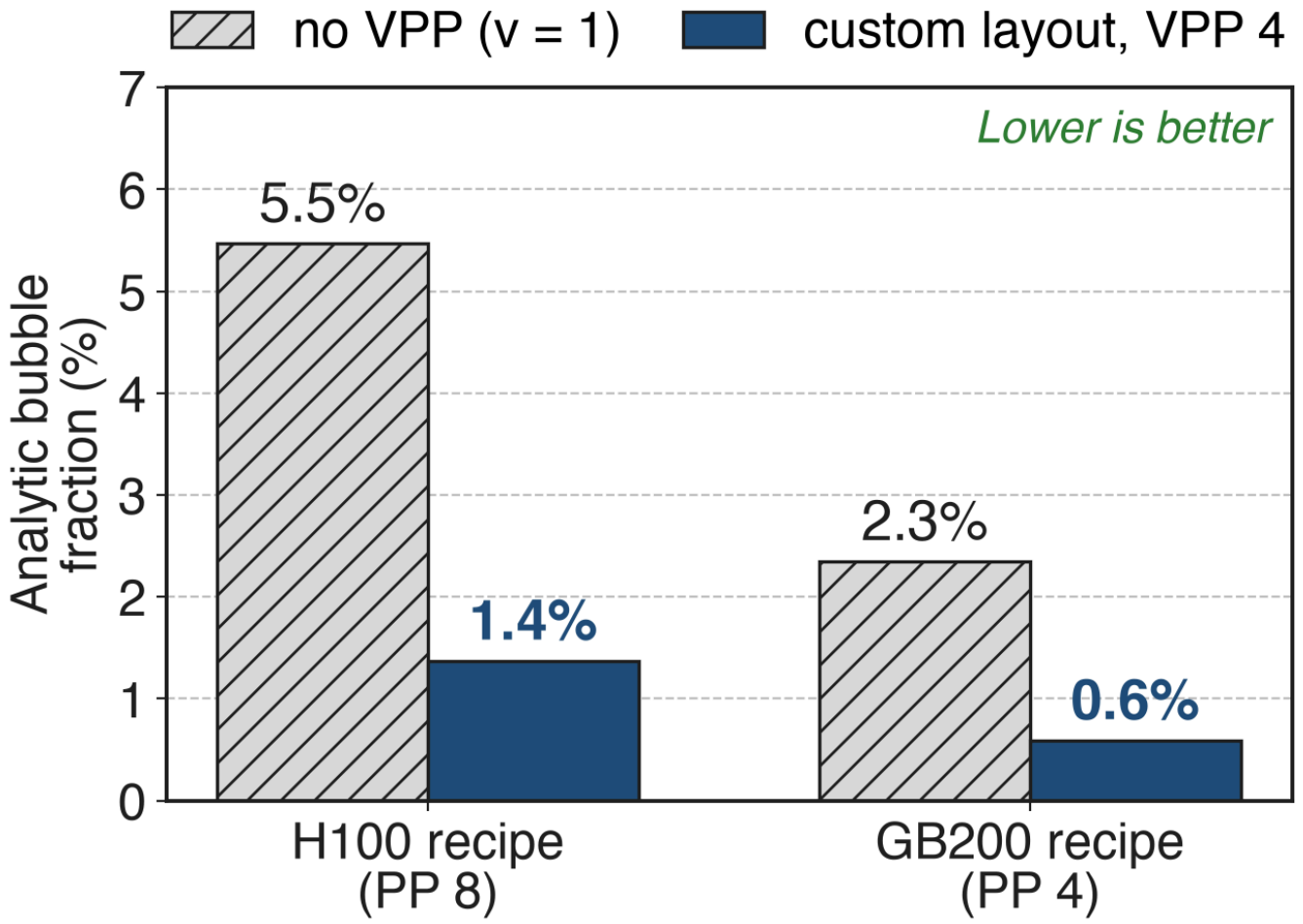
- **Placement (by hand)**
- E + 3 dense: about 2 MoE
- MTP alone; loss alone

- **Validator checks**
- E first, L last, once each
- MTP in its rank's last chunk

The layout is checked against num_layers and mtp_num_layers before training starts

VPP4 cuts the analytic bubble 4×: 5.5% → 1.4%

- DeepSeek-V3 recipes: m = **128** micro-batches
- PTD-P model, balanced stages (analytic)
- Interleaving costs **v×** more P2P traffic



Both DeepSeek-V3 recipes ship custom layouts

- 1,024 H100, PP 8: `Et*3|(tt|)*29m|L`: **368** TFLOPS/GPU
- 256 GB200, PP 4: `Et*4|(tttt|)*14tmL`: **1,048** TFLOPS/GPU
- VPP also unlocks 1F1B EP overlap when PP > 1 (deck 11)

The cost: balance by hand, v× more P2P

- No cost model: you write the balance
- Larger VPP: more P2P traffic
- Flag marked experimental
- MTP and encoder restrictions
- Excludes the other split flags
- FSDP + EP avoids PP design (deck 70)

Summary

- Uniform VPP needs equal stages: DeepSeek-V3's **61** layers cannot split
- **Insight**: a pipeline is a list of stages, so write the list
- One flag: `--pipeline-model-parallel-layout`; VPP = stages / PP
- H100 recipe: bubble **5.5%** → **1.4%** (analytic); **368** TFLOPS/GPU stack

https://arxiv.org/abs/2603.07685